Forelesning 21
To-dimensjonale randverdiproblemer og varmeligningen i 2D

Kompendium §3.3.5–§3.3.6

IMAx3012 / IMAA3012 – Matematikk 3B

I dag

- **Forrige gang (F20):** endelige differanser i *1D* – Dirichlet- og Neumann-RVP, Jacobi/Gauss-Seidel.
- **I dag:** samme ideer i *2D*, pluss vårt første *initialverdiproblem* – varmeligningen:
  1. To-dimensjonale randverdiproblemer: Laplace- og Poisson-ligningen (§3.3.5)
  2. Varmeligningen: semi-diskretisering og eksplisitt Euler, i 1D og 2D (§3.3.6)
- **Neste gang (F22):** leapfrog-metoden for bølgeligningen – en annen strategi for tidsavhengige PDE-er.

Hva er egentlig nytt i dag?

Definisjon: Randverdiproblem (alt så langt)

- Betingelser i *begge* ender / på *hele* randen.
- Alle ukjente koblet $\Rightarrow$ løses *samlet* (ett ligningssystem).
- F20 (1D) og 2D Laplace/Poisson i dag.

Definisjon: Initialverdiproblem (nytt!)

- Betingelse kun i *én* ende ($t = 0$).
- Løses *steg for steg*: vi marsjerer fremover.
- Ny utfordring: *stabilitet* – krav til tidssteget, $\Delta t/\Delta x^2 \leq \frac{1}{2}$.

- Overgangen 1D $\rightarrow$ 2D (ODE $\rightarrow$ PDE) endrer *ikke* strategien – systemet blir bare større.
- Varmeligningen = **RVP i** x + **IVP i** t (initial-randverdiproblem).
- Det er *hvor betingelsene står* som teller, ikke hva variabelen heter: F20-ligningen med $u(0)$ og $u'(0)$ gitt ville vært et IVP i x.

Poisson- og Laplace-ligningen i 2D

Vi generaliserer 1D-RVP-et til to romdimensjoner. Poisson-ligningen (Laplace: spesialtilfellet $f = 0$):

$$\Delta u = u_{xx} + u_{yy} = f(x, y) \quad \text{i } D, \qquad u = g \quad \text{på } \partial D,$$

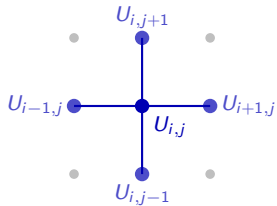
på et rektangel $D = [a, b] \times [c, d]$, med gitte Dirichlet-randverdier g.

- Akkurat som i 1D: diskretiser domenet, approksimer deriverte, løs ligningssystemet.
- Nytt: randen ∂D er nå en hel *kurve* (de fire kantene av rektangelet), ikke bare to endepunkter.

Diskretisering: fempunktsformelen

Rutenett $x_i = a + i\Delta x$, $y_j = c + j\Delta y$, $U_{i,j} \approx u(x_i, y_j)$. Sentraldifferanser i hver retning:

$$\frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{\Delta x^2} + \frac{U_{i,j-1} - 2U_{i,j} + U_{i,j+1}}{\Delta y^2} = f(x_i, y_j).$$



Kalles *fempunktsformelen* (eng. five-point stencil): kobler hvert punkt til sine fire nærmeste naboer.
I spesialtilfellet $\Delta x = \Delta y = h$:

$$U_{i-1,j} + U_{i+1,j} + U_{i,j-1} + U_{i,j+1} - 4U_{i,j} = h^2 f(x_i, y_j). \tag{1}$$

Løsning: hvorfor ikke bare $A\mathbf{U} = \mathbf{b}$ direkte?

I prinsippet: samle alle $U_{i,j}$ i én lang vektor (f.eks. radvis), sett opp fempunktsformelen som et lineært system $A\mathbf{U} = \mathbf{b}$, og løs direkte – akkurat som i 1D.

Merk: Hvorfor er dette tungvint i 2D?

A blir en stor matrise (dimensjon = antall rutenettpunkter i hver retning, multiplisert sammen), og den eksplisitte oppbyggingen krever mye mer bokholderi enn i 1D. For store 2D-problemer er det upraktisk å sette opp hele A.

Løsning: bruk Jacobi- eller Gauss-Seidel-iterasjon fra F20 – disse generaliserer direkte, og unngår å bygge opp A i det hele tatt.

Jacobi/Gauss-Seidel i 2D

Løs fempunktsformelen for $U_{i,j}$:

$$U_{i,j} = \frac{1}{4}\Big(U_{i-1,j} + U_{i+1,j} + U_{i,j-1} + U_{i,j+1} - h^2 f(x_i, y_j)\Big), \tag{2}$$

og bruk dette som oppdateringsformel, akkurat som i 1D:

- **Jacobi:** oppdater alle $U_{i,j}$ samtidig, med verdier fra forrige iterasjon.
- **Gauss-Seidel:** bruk oppdaterte verdier så snart de er tilgjengelige (f.eks. rad for rad).

Randpunktene $U_{i,j}$ holdes fast lik de gitte Dirichlet-verdiene $g(x_i, y_j)$ gjennom hele iterasjonen – kun indre punkter oppdateres.

Merk: Posisjonsavhengige randbetingelser

g trenger ikke være konstant – den kan variere langs hver vegg. Det eneste som kreves er at vi kan evaluere $g(x_i, y_j)$ i hvert randpunkt.

Implementering (F21-notatbok)

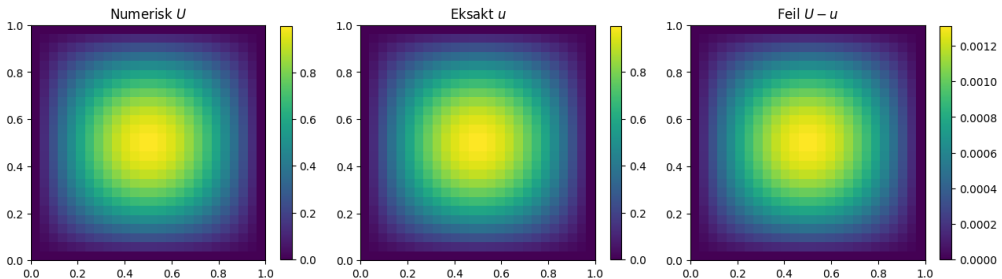
Python – setter Dirichlet-rand og itererer

```
def sett_rand(U, g, x, y):
    U[0, :]  = [g(xi, y[0])  for xi in x]
    U[-1, :] = [g(xi, y[-1]) for xi in x]
    U[:, 0]  = [g(x[0],  yj) for yj in y]
    U[:, -1] = [g(x[-1], yj) for yj in y]
    return U

def jacobi_poisson2d(f, g, Lx, Ly, Nx, Ny, tol=1e-8, maxit=200_000):
    # ... sett opp rutenett, initialiser U, sett rand med sett_rand ...
    # itererer U[i,j] = 0.25*(U[i-1,j]+U[i+1,j]+U[i,j-1]+U[i,j+1] - h^2 f)
    ...
```

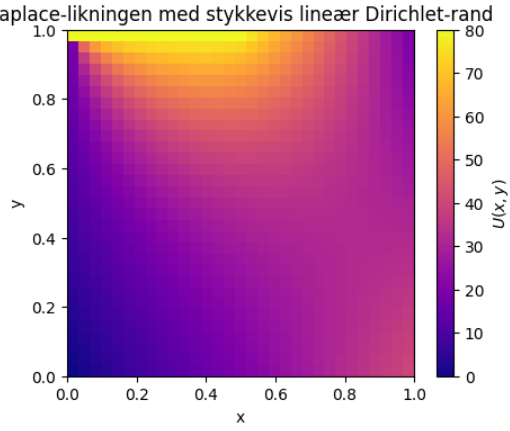
Test mot kjent løsning

Manufactured solution: $u(x, y) = \sin(\pi x)\sin(\pi y)$ på $[0, 1]^2$, $u = 0$ på randen, $f = \Delta u = -2\pi^2 \sin(\pi x)\sin(\pi y)$.



Posisjonsavhengig (stykkevis) Dirichlet-rand

Samme kode, men $g(x, y)$ er nå stykkevis lineær langs hver av de fire kantene – akkurat som i øvingsoppgaver med f.eks. en temperaturfordeling langs randen av en plate.



Fra randverdiproblem til initial-randverdiproblem

Så langt: kun *randverdiproblemer* (1D og 2D). Nå: et problem som er et randverdiproblem i x, men et *initialverdiproblem* i t – med varmeligningen som eksempel (men teknikken gjelder generelt):

$$u_t = u_{xx}, \quad 0 \leq x \leq 1, \qquad u(0,t) = g_0(t), \; u(1,t) = g_1(t), \qquad u(x,0) = f(x).$$

Essensen: *semi-diskretisering* – diskretiser kun i rom, la tiden foreløpig være kontinuerlig.

Semi-diskretisering: steg 1–3

1. Diskretiser kun i x: $x_i = i\Delta x$, $\Delta x = 1/M$. For hver x_i er $u(x_i, t)$ en funksjon *kun* av t.
2. Diskretiser høyresiden med sentraldifferanse (i rom, ved et vilkårlig fast t):

$$\frac{\partial u}{\partial t}(x_i, t) \approx \frac{u(x_{i+1}, t) - 2u(x_i, t) + u(x_{i-1}, t)}{\Delta x^2}.$$

3. Erstatt $u(x_i, t)$ med $U_i(t)$:

$$U_i'(t) = \frac{U_{i+1}(t) - 2U_i(t) + U_{i-1}(t)}{\Delta x^2}, \quad i = 1, \ldots, M-1,$$

   med $U_0(t) = g_0(t)$, $U_M(t) = g_1(t)$, $U_i(0) = f(x_i)$.

Dette er et veldefinert **system av ODE-er** – en *semi-diskretisering* av PDE-en.

Viktig presisering: hva bestemmer ordenen i tid?

Merk: Semi-diskretisering sier ingenting om tidsordenen ennå

Steg 1–3 diskretiserer kun i *rom*. Resultatet er et system av ODE-er som i prinsippet er *eksakt* i tid – ingen tidsapproksimasjon er gjort ennå. Sentraldifferansen i rom er $O(\Delta x^2)$, men dette sier ingenting om nøyaktigheten i tid.
Nøyaktigheten i *tid* avgjøres først i **steg 4**, når vi velger en konkret metode for å løse ODE-systemet.

Steg 4: løs ODE-systemet – eksplisitt Euler

For konkrethets skyld bruker vi eksplisitt Euler med tidssteg Δt:

$$U_i^{n+1} = U_i^n + r(U_{i+1}^n - 2U_i^n + U_{i-1}^n), \qquad r = \frac{\Delta t}{\Delta x^2}, \tag{3}$$

der $U_i^n \approx u(x_i, t_n)$, $t_n = n\Delta t$.

Merk: Eksplisitt Euler er kun 1. ordens i tid

Selv om romdiskretiseringen (sentraldifferansen) er $O(\Delta x^2)$, er *eksplisitt Euler selv* en **førsteordens** metode i tid – akkurat som for ODE-er. Den totale feilen i dette skjemaet er dermed $O(\Delta t) + O(\Delta x^2)$: førsteordens i tid, andreordens i rom.

Hvorfor stabilitet nå? Fordi vi marsjerer

- RVP: alt løses *samtidig* – ingen steg der feil kan hope seg opp.
- IVP: hvert steg bygger på det forrige. Feil forsterket med G per steg $\Rightarrow$ G^n etter n steg. $|G| > 1 \Rightarrow$ eksplosjon.

Oppsummering: Sagtann-feil: $e_i^n = (-1)^i \varepsilon$ (verst tenkelige)

Siden $e_{i\pm1}^n = -e_i^n$:

$$e_i^{n+1} = e_i^n + r(e_{i+1}^n - 2e_i^n + e_{i-1}^n) = (1 - 4r)\, e_i^n.$$

Forsterkning $G = 1 - 4r$ per steg, og

$$|1 - 4r| \leq 1 \quad \Longleftrightarrow \quad r = \frac{\Delta t}{\Delta x^2} \leq \frac{1}{2}.$$

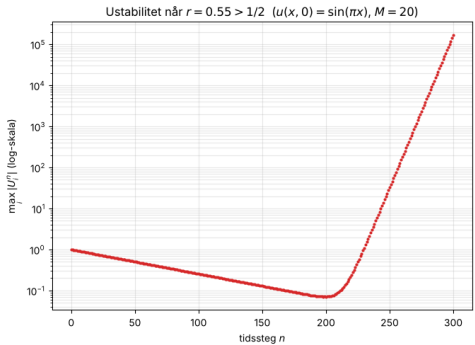
Stabilitet: CFL-betingelsen

Merk: Stabilitet

Eksplisitt Euler er *ikke* ubetinget stabil: for stor Δt i forhold til Δx gjør at feil vokser eksponentielt. Metoden er stabil nøyaktig når

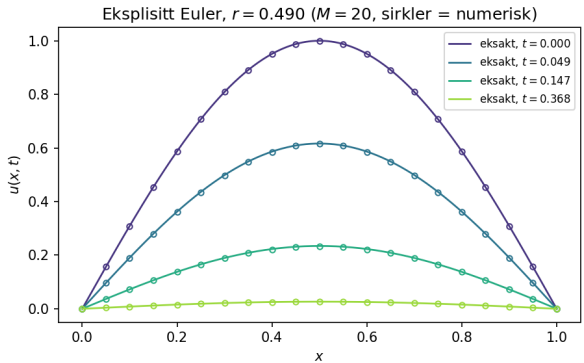
$$r = \frac{\Delta t}{\Delta x^2} \leq \frac{1}{2}.$$

Dette er et eksempel på en *CFL-betingelse* (Courant–Friedrichs–Lewy). I praksis: finere romdiskretisering (Δx mindre) krever et *mye* mindre tidssteg, siden $\Delta t \propto \Delta x^2$.



Numerisk eksempel 1: kjent eksakt løsning

$u(x,0) = \sin(\pi x)$, $g_0 = g_1 = 0$. Eksakt løsning: $u(x,t) = e^{-\pi^2 t}\sin(\pi x)$.

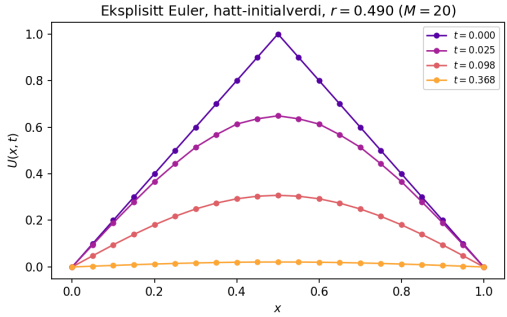


Sirkler = numerisk ($r < 1/2$, stabilt), heltrukket = eksakt – de faller sammen, og begge går mot 0 når $t \to \infty$ (varmen jevner seg ut).

Numerisk eksempel 2: hatt-funksjon

$$u(x,0) = \begin{cases} 2x, & 0 \le x \le 0.5 \\ 2(1-x), & 0.5 < x \le 1 \end{cases}, \qquad g_0 = g_1 = 0.$$

Her er *ikke* den eksakte løsningen gitt i enkel form – vi observerer bare den numeriske løsningen glatte seg ut over tid.



Generalisering til to romdimensjoner

Samme teknikk på $u_t = \Delta u = u_{xx} + u_{yy}$, med fempunktsformelen i rom og eksplisitt Euler i tid, $\Delta x = \Delta y = h$:

$$U_{i,j}^{n+1} = U_{i,j}^n + r\Big(U_{i-1,j}^n + U_{i+1,j}^n + U_{i,j-1}^n + U_{i,j+1}^n - 4U_{i,j}^n\Big), \quad r = \frac{\Delta t}{h^2}, \tag{4}$$

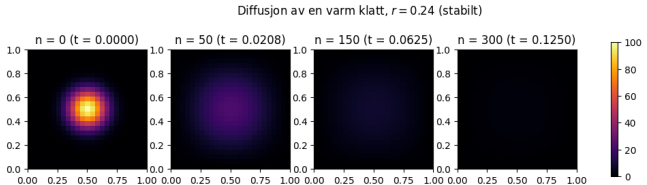
på indre punkter; randpunktene holdes fast lik gitte randverdier i hvert tidssteg.

Merk: Stabilitet i 2D

Stabil når $r \leq \frac{1}{4}$ (og mer generelt $\frac{\Delta t}{\Delta x^2} + \frac{\Delta t}{\Delta y^2} \leq \frac{1}{2}$ hvis $\Delta x \neq \Delta y$) – strengere enn i 1D.

2D varmeligning: stabil kjøring

En "varm klatt" i midten av en kald plate, $r = 0.24 < 1/4$:



Merk: Kobling til Laplace/Poisson

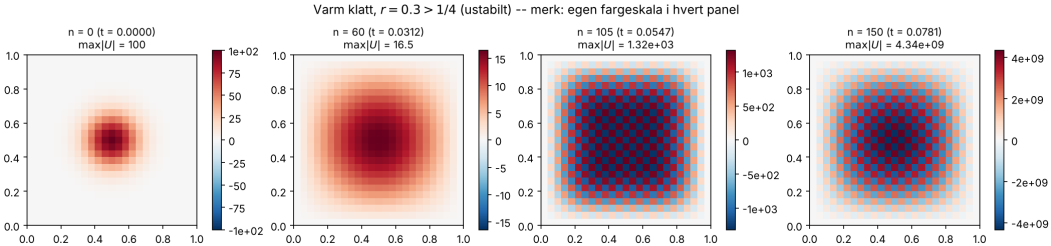
Når løsningen slutter å endre seg ($U^{n+1} \approx U^n$), er $u_t = 0$, altså $\Delta u = 0$: den stasjonære løsningen *er* Laplace-løsningen med samme randverdier. Med $r = \frac{1}{4}$ blir skjemaet

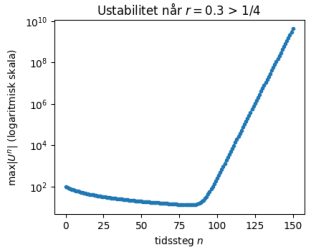
$$U_{i,j}^{n+1} = \tfrac{1}{4}\big(U_{i-1,j}^n + U_{i+1,j}^n + U_{i,j-1}^n + U_{i,j+1}^n\big)$$

– nøyaktig **Jacobi**! Jacobi-iterasjon = eksplisitt Euler med største stabile tidssteg.

2D varmeligning: ustabil kjøring

Samme problem, men $r = 0.3 > 1/4$ – skjemaet "eksploderer":





- Først ser alt fint ut ($n = 60$): klatten flyter utover.
- Men avrundingsfeil i *sjakkbrett*-moden $(-1)^{i+j}$ forsterkes med $|G| = |1 - 8r| = 1.4$ per steg.
- Ved $n \approx 100$ har sjakkbrettet tatt over, og $\max |U|$ vokser eksponentielt.

Oppsummering

- 2D Poisson/Laplace: fempunktsformelen + Jacobi/Gauss-Seidel-iterasjon, direkte generalisering fra 1D. Randen kan være posisjonsavhengig.
- Varmeligningen løses ved *semi-diskretisering*: diskretiser i rom $\rightarrow$ ODE-system $\rightarrow$ velg en tidsintegrator.
- Semi-diskretiseringen i seg selv sier ingenting om tidsordenen – det er *valget av ODE-metode* (her: eksplisitt Euler) som gjør skjemaet førsteordens i tid.
- Eksplisitt Euler er betinget stabilt: $r = \Delta t/\Delta x^2 \leq \frac{1}{2}$ i 1D, $r \leq \frac{1}{4}$ i 2D.

Neste gang: F22 – leapfrog-metoden

Neste forelesning

Varmeligningen er *førsteordens i tid*, så eksplisitt Euler (ett tidsnivå) er en naturlig start. Bølgeligningen $u_{tt} = c^2 u_{xx}$ er derimot *andreordens i tid* – eksplisitt Euler passer ikke direkte.

Neste gang ser vi på **leapfrog-metoden**: en sentraldifferanse også i tid, som er *bedre egnet* for denne typen problemer – andreordens nøyaktig i tid, og uten den kunstige numeriske dempingen eksplisitt Euler ville gitt.